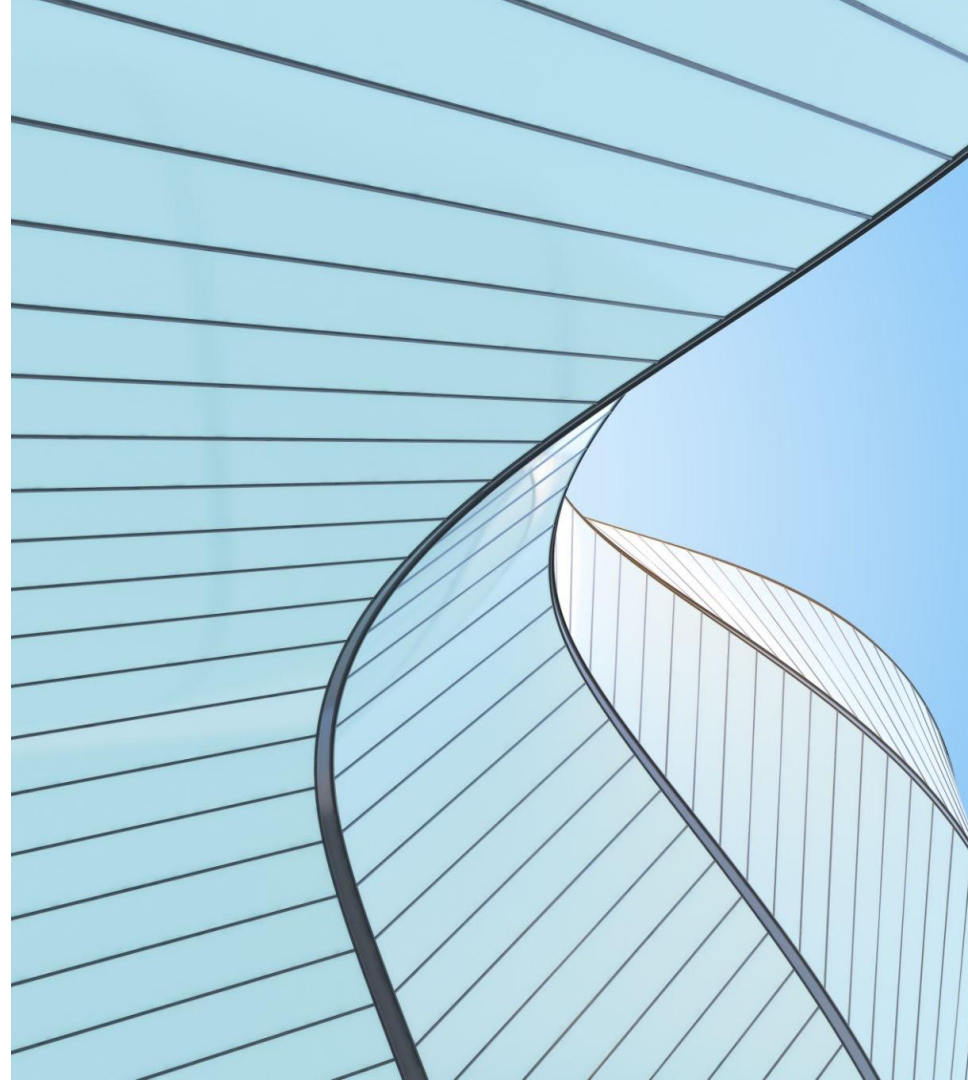


Company Name

Date

Ensemble learning Tutorial by python

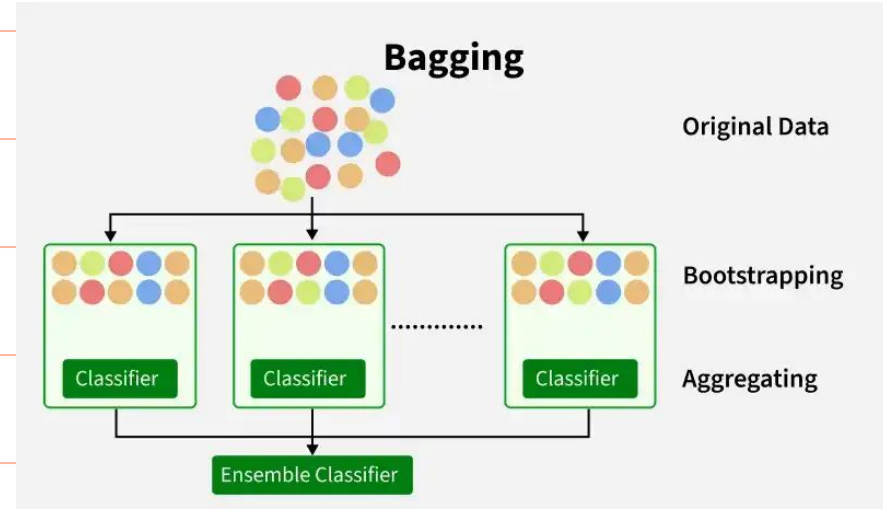
By Eng. Radwa Taher



Bagging vs Boosting in Machine Learning

- **Bagging** and **boosting** are both ensemble learning techniques used to improve model performance by combining multiple models.
- The main difference is that
 - Bagging reduces **variance** by training models independently.
 - Boosting reduces **bias** by training models sequentially, focusing on previous errors.

Bagging

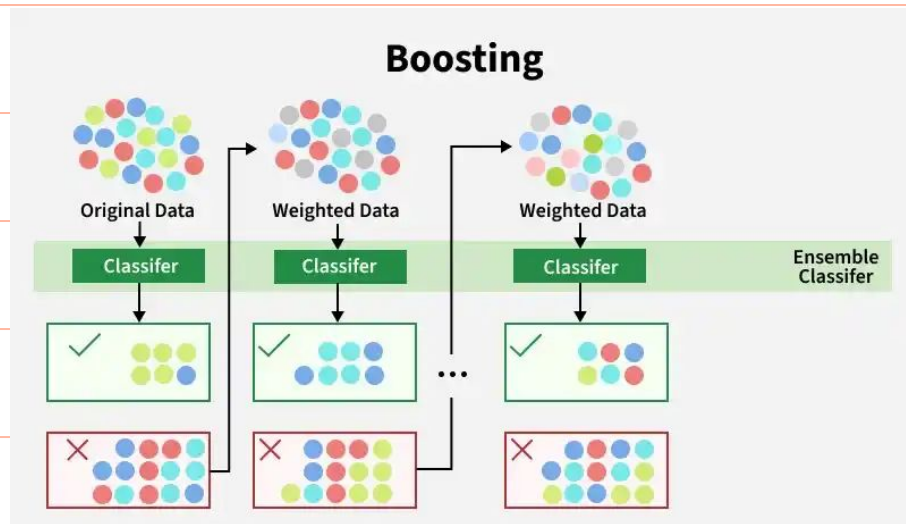


Bagging builds independent, parallel models (e.g., Random Forest) on random data subsets to reduce variance.

Bagging

- Uses random sampling with replacement
- Trains models independently
- Combines predictions using voting or averaging
- Reduces overfitting
- Used in Random Forest, Credit risk prediction and Medical diagnosis systems

Boosting



Boosting focuses on improving model accuracy by training models sequentially. Each new model pays more attention to the data points that were misclassified by previous models. Over time, the ensemble becomes better at handling difficult cases.

Boosting

Boosting is effective for reducing bias and works well even with weak learners.

- Trains models sequentially
- Gives more weight to misclassified samples
- Combines models using weighted voting
- Reduces bias
- Used in Fraud detection

Difference Between Bagging and Boosting

Feature	Bagging	Boosting
Training style	Trains independent models	Trains sequential models
Main goal	Reduce variance	Reduce bias
Handling errors	All samples treated equally	Focuses on misclassified samples
Overfitting	It is less sensitive to overfitting	More prone to overfitting
Parallel training	Yes it supports parallel computing	No it does not support parallel computing
Sensitivity to noise	It is not affected by noise and outliers	Sensitive to noise and outliers

Import libraries

```
import numpy as np  
import pandas as pd
```

```
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import accuracy_score, roc_auc_score,  
    confusion_matrix, classification_report  
from sklearn.preprocessing import StandardScaler  
from sklearn.pipeline import Pipeline
```


Import libraries

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier,
AdaBoostClassifier, GradientBoostingClassifier, StackingClassifier
from sklearn.svm import SVC

import matplotlib.pyplot as plt

SEED = 42 # SEED = 42 means Purpose
# Fixing the randomness Getting the same results every time
```

1) Load data and split

```
data = load_breast_cancer(as_frame=True)
```

```
X = data.data
```

```
y = data.target # 0/1
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=SEED, stratify=y  
)
```

```
X_train.shape, X_test.shape, y_train.mean(), y_test.mean()
```

2) A simple baseline model

- We start with **logistic regression** (a strong baseline for many tabular binary datasets).
- Because **logistic regression** works best with scaled features, we use a **pipeline**:
StandardScaler -> LogisticRegression.

```
▶ baseline = Pipeline([  
    ("scaler", StandardScaler()),  
    ("clf", LogisticRegression(max_iter=5000, random_state=SEED))  
    # The maximum number of training iterations (steps) to ensure  
    #the model converges properly.  
])
```

2) A simple baseline model

```
baseline.fit(X_train, y_train)
class_1_prob = baseline.predict_proba(X_test)[:, 1]
#The probability that each test sample belongs to class 1
# Give probability for drawing Area Under Curve
y_pred = (class_1_prob >= 0.5).astype(int)
# Convert probability into binary 0 ,1

print("Accuracy:", accuracy_score(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, class_1_prob))
print("\nConfusion matrix:\n", confusion_matrix(y_test, y_pred))
```

Definition of roc_auc_score()

- **roc_auc_score()** is a function in **scikit-learn** that computes the **ROC-AUC score**, which measures how well a classification model can distinguish between two classes.
- **What does it mean??**

The ROC-AUC score represents the **Area Under the ROC Curve**, where the ROC curve plots: **True Positive Rate (TPR)** vs **False Positive Rate (FPR)**

Definition of roc_auc_score()

Interpretation

roc_auc_score() tells you:

good the model is at ranking positive examples higher than negative ones.

In other words: A higher score means the model separates the classes better

Score range

- **1.0** → Perfect classifier, **0.5** → Random guessing, **< 0.5** → Model is performing worse than random

Mathematical meaning The ROC-AUC score is equal to the probability that:

A randomly chosen positive sample receives a higher predicted score than a randomly chosen negative sample.

3) Bagging

Bagging trains many copies of the same model on bootstrapped samples.

We'll use a **decision tree** as the base model.

- A single tree can be unstable.
- Bagging tends to improve stability and accuracy.

We will use `BaggingClassifier` (sklearn) to keep it simple.

```
base_tree = DecisionTreeClassifier(random_state=SEED)
```

Bagging Classifier

```
base_tree = DecisionTreeClassifier(random_state=SEED)

bagging = BaggingClassifier(
    estimator=base_tree,
    # the base model repeated many times (e.g., Decision Tree)
    n_estimators=200, # number of models (200)
    bootstrap=True,
    # train each model on a bootstrap sample
    random_state=SEED,
    n_jobs=-1 # use all CPU cores for faster training
)
```


Bagging Classifier

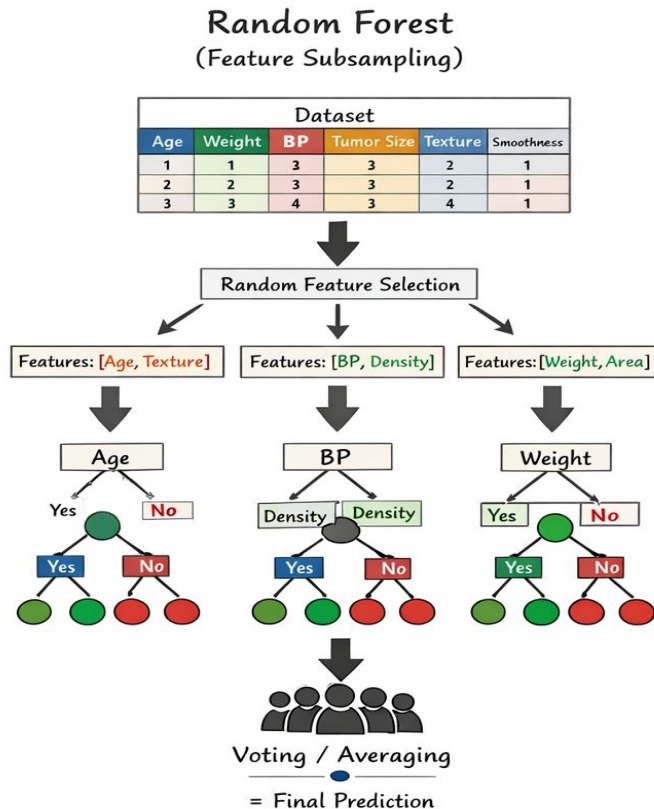
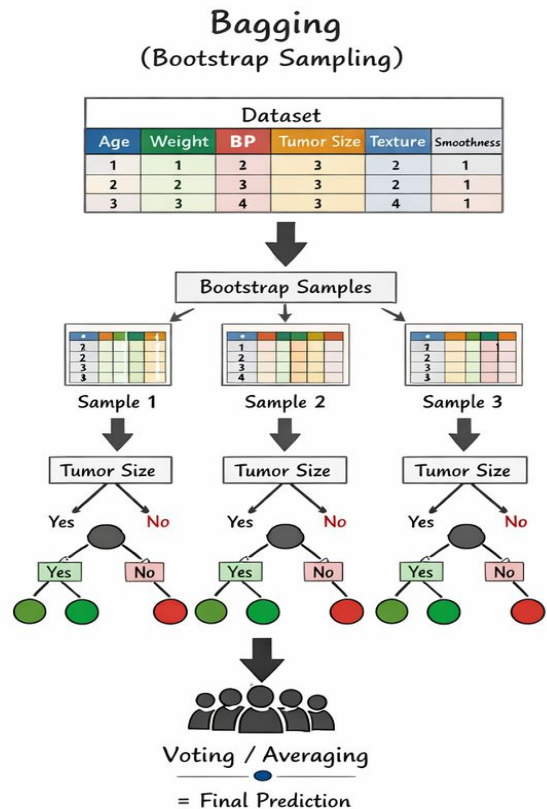
```
bagging.fit(X_train, y_train)
class_1_prob = bagging.predict_proba(X_test)[:, 1]
y_pred = (class_1_prob >= 0.5).astype(int)

print("Bagging Accuracy:", accuracy_score(y_test, y_pred))
print("Bagging ROC-AUC:", roc_auc_score(y_test, class_1_prob))
```

Bagging Accuracy: 0.951048951048951

Bagging ROC-AUC: 0.9932914046121593

A **random forest** is basically bagging plus extra *randomness* (feature subsampling).



Number of Random Features

✓ What is `max_features` in Random Forest?

`max_features` controls **how many features (columns)** the model is allowed to consider **when making each split** in a decision tree.

Python

```
X.shape[1] = 20 features
```

Case 1: `max_features=5`

Python

```
rf = RandomForestClassifier(max_features=5)
```

- At each split, the tree randomly selects **5 out of 20 features**, then chooses the best split among those 5.

Number of Random Features

Common options for `max_features`

1) `"sqrt"` (default for classification)

Python

```
rf = RandomForestClassifier(max_features="sqrt")
```

Uses:

$$\sqrt{20} \approx 4$$

So each split considers about **4 features**.

Number of Random Features

2) "log2"

Python

```
rf = RandomForestClassifier(max_features="log2")
```

Uses:

$$\log_2(20) \approx 4$$

Also around **4 features**.

Number of Random Features

3) None (use all features)

Python

```
rf = RandomForestClassifier(max_features=None)
```

Each split considers all **20 features**.

More powerful splits, but trees become more similar → higher risk of overfitting.

Random Forest

[+ Code](#)[+ Text](#)

```
rf = RandomForestClassifier(n_estimators=300, random_state=SEED, n_jobs=-1)
# Number of Trees
rf.fit(X_train, y_train)
p_rf = rf.predict_proba(X_test)[: , 1]
y_hat_rf = (p_rf >= 0.5).astype(int)

print("Random Forest Accuracy:", accuracy_score(y_test, y_hat_rf))
print("Random Forest ROC-AUC:", roc_auc_score(y_test, p_rf))
```

```
Random Forest Accuracy: 0.958041958041958
Random Forest ROC-AUC: 0.9948637316561845
```

Boosting

- **Boosting** trains models one by one, each trying to correct mistakes made before.
- We'll use **AdaBoost** with small decision trees ("stumps") as weak learners.

```
stump = DecisionTreeClassifier(max_depth=1, random_state=SEED)

adaboost = AdaBoostClassifier(
    estimator=stump,
    #The weak learner that is repeated (Decision Stump)
    n_estimators=200,
    # The number of weak learners (200)
    learning_rate=0.5,
    #The contribution (impact) of each model to the final prediction
    random_state=SEED
)
```


1] AdaBoost (Adaptive Boosting)

How it works:

1. Train a weak model (usually a stump = 1-level tree)
2. Increase weight of misclassified samples
3. Train the next model on weighted data
4. Combine all learners with weighted voting

Key feature:

✓ Very sensitive to noisy data and outliers
Because outliers keep getting higher weights.

AdaBoost

```
adaboost.fit(X_train, y_train)
p_ada = adaboost.predict_proba(X_test)[: , 1]
y_hat_ada = (p_ada >= 0.5).astype(int)

print("AdaBoost Accuracy:", accuracy_score(y_test, y_hat_ada))
print("AdaBoost ROC-AUC:", roc_auc_score(y_test, p_ada))
```

AdaBoost Accuracy: 0.958041958041958

AdaBoost ROC-AUC: 0.9878406708595389

AdaBoost

What is the default estimator?

- For classification: a **DecisionTreeClassifier** stump (a decision tree with `max_depth=1`)

Python

```
ada = AdaBoostClassifier(n_estimators=100)
```

You didn't pass `estimator=...`, so AdaBoost uses the default. $n_estimators = 50$

What is the default estimator?

- For classification: a **DecisionTreeClassifier** stump (a decision tree with `max_depth=1`).

Gradient Boosting

- ◆ **Main idea:**

Gradient Boosting fits new learners to the **residual errors** (the difference between prediction and truth).

Instead of changing sample weights, it learns using gradients (like optimization).

Key feature:



More flexible and powerful than AdaBoost

Less sensitive to outliers (depending on loss)

Gradient Boosting

```
▶ gb = GradientBoostingClassifier(random_state=SEED)
# By default DecisionTreeClassifier(max_depth=3)
gb.fit(X_train, y_train)
p_gb = gb.predict_proba(X_test)[:, 1]
y_hat_gb = (p_gb >= 0.5).astype(int)

print("GradBoost Accuracy:", accuracy_score(y_test, y_hat_gb))
print("GradBoost ROC-AUC:", roc_auc_score(y_test, p_gb))
```

```
... GradBoost Accuracy: 0.958041958041958
    GradBoost ROC-AUC: 0.9924528301886792
```

5) Stacking

Stacking combines different kinds of models, then trains a **meta-model** to learn the best combination.

Steps:

1. Train several base models
2. Create their predictions
3. Train a meta-model on those predictions (sklearn handles the details)

We'll use:

- Logistic Regression
- SVM (with scaling)
- Random Forest

Meta-model: Logistic Regression

Stacking

```
estimators = [  
    ("lr", Pipeline([("scaler", StandardScaler()),  
                      ("clf", LogisticRegression(max_iter=5000, random_state=SEED))])),  
    ("svm", Pipeline([("scaler", StandardScaler()),  
                       ("clf", SVC(probability=True, random_state=SEED))])),  
    ("rf", RandomForestClassifier(n_estimators=250, random_state=SEED, n_jobs=-1)),  
]  
  
stack = StackingClassifier(  
    estimators=estimators,  
    final_estimator=LogisticRegression(max_iter=5000, random_state=SEED),  
    stack_method="predict_proba",  
    n_jobs=-1  
)
```

Stacking

:Meta Model ∪ New dataset			
Target	RF_pred	SVM_pred	LR_pred
1	0.90	0.70	0.80
0	0.20	0.40	0.30

Stacking

```
stack.fit(X_train, y_train)
p_stack = stack.predict_proba(X_test)[: , 1]
y_hat_stack = (p_stack >= 0.5).astype(int)

print("Stacking Accuracy:", accuracy_score(y_test, y_hat_stack))
print("Stacking ROC-AUC:", roc_auc_score(y_test, p_stack))
```

Comparing them all

```
rows = [  
    ("Baseline (LogReg)", p_base),  
    ("Bagging (Trees)", p_bag),  
    ("Random Forest", p_rf),  
    ("AdaBoost", p_ada),  
    ("Gradient Boosting", p_gb),  
    ("Stacking", p_stack),  
]
```

Comparing them all

```
summary = []
for name, p in rows:
    summary.append({
        "model": name,
        "accuracy": accuracy_score(y_test, (p >= 0.5).astype(int)),
        "roc_auc": roc_auc_score(y_test, p),
    })

pd.DataFrame(summary).sort_values("roc_auc", ascending=False)
```

Comparing them all

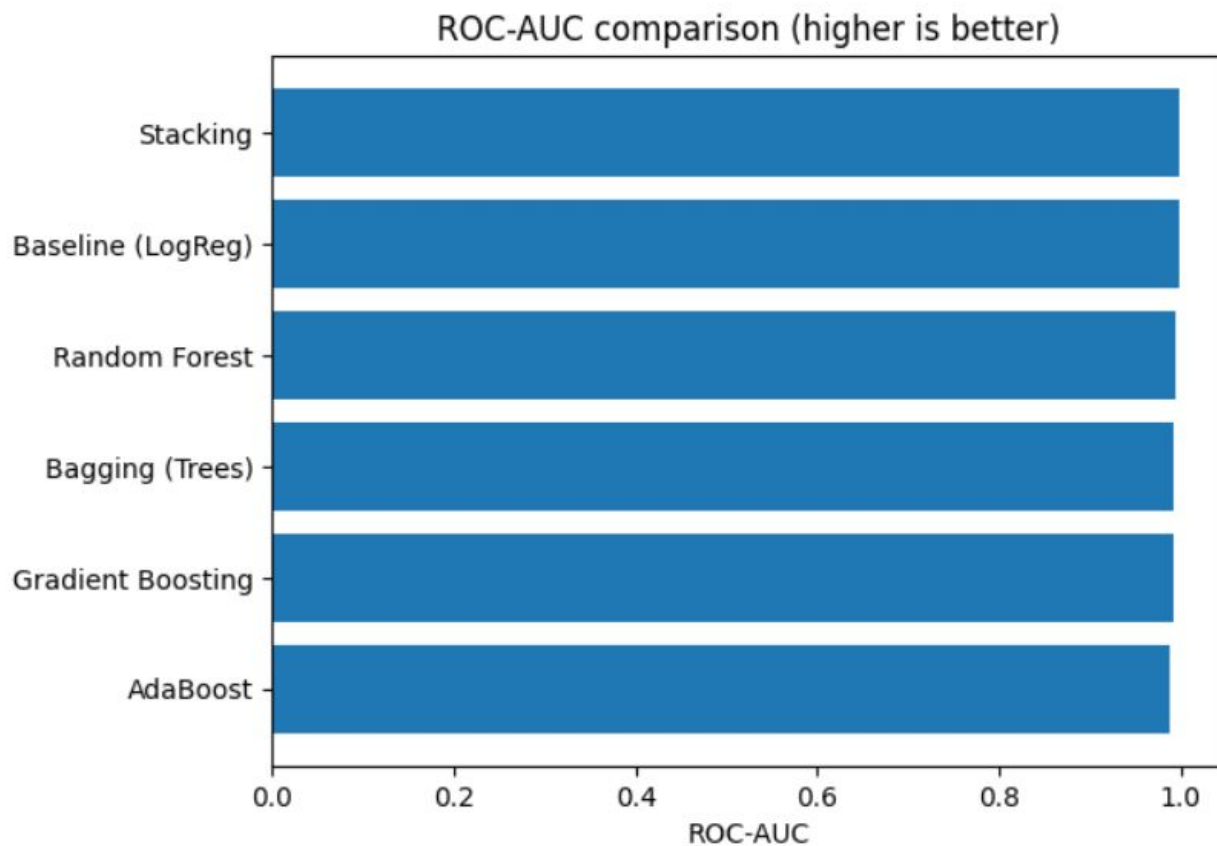
	model	accuracy	roc_auc
5	Stacking	0.979021	0.998323
0	Baseline (LogReg)	0.986014	0.997694
2	Random Forest	0.958042	0.994864
1	Bagging (Trees)	0.951049	0.993291
4	Gradient Boosting	0.958042	0.992453
3	AdaBoost	0.958042	0.987841

Comparing them all

```
df = pd.DataFrame(summary).sort_values("roc_auc", ascending=True)

plt.figure()
plt.barh(df["model"], df["roc_auc"])
plt.xlabel("ROC-AUC")
plt.title("ROC-AUC comparison (higher is better)")
plt.show()
```

Comparing them all



Exercises

1. Change `n_estimators` in bagging and boosting (try 10, 50, 200, 500). What happens?
2. Change `learning_rate` in AdaBoost (0.1, 0.5, 1.0). What happens?
- 3.
4. Change the stacking base models (add kNN, remove SVM, etc.). Does stacking improve?

Write down a Report